

Rust Voice-to-Text (“Ottex”) App for Windows

We aim to build a Windows app in Rust that mimics Ottex.ai’s voice-to-clipboard typing. In essence, the app sits in the tray, listens for a global hotkey (e.g. Alt+Space) to start/stop recording, sends the audio to a speech-to-text API (Google STT preferred, OpenRouter/Whisper fallback), then **simulates typing** the resulting text into the active window. Secondary features include a “Raycast-style” overlay for quick AI shortcuts (e.g. “Fix Grammar” on selected text).

For example, Ottex’s website touts itself as *“the voice-to-text AI that turns speech into clear, polished writing in every app”* ¹. We will replicate that core flow on Windows using Rust: capture mic audio via **cpal**, encode it, send to a transcription API, then use **enigo** to paste/type the text ². The following sections detail the stack and architecture.

1. Tech Stack: Crates and Tools

- **System Tray (tray-icon)** – Create a native tray icon/app menu. The `tray-icon` crate “lets you create tray icons for desktop applications” on Windows (and macOS/Linux) ³. We’ll use it for a minimalist UI/menus.
- **Global Hotkey (global_hotkey)** – Register a system-wide hotkey. The `global_hotkey` crate lets us bind keys like Alt+Space even when our app isn’t focused (Windows/macOS/Linux) ⁴. On key press/release we start/stop recording.
- **Audio Capture (cpal)** – For low-level audio I/O we use `cpal`. It provides cross-platform, low-level audio streams (input/output). We’ll open a default input stream and capture raw PCM samples into a buffer.
- **WAV Encoding (hound)** – To send to transcription APIs, audio usually must be in a common format (WAV, MP3, etc.). We can encode the captured PCM to WAV using the `hound` crate, “a WAV encoding and decoding library” ⁵.
- **HTTP Client (reqwest)** – To call cloud APIs (Google or OpenAI), we need HTTP. The `reqwest` crate is a convenient HTTP(S) client ⁶. We’ll use it to POST audio to the chosen API endpoint.
- **Input Simulation (enigo)** – After transcription, we simulate keystrokes to “type” the text into the current app. The `enigo` crate lets us send keyboard events as if from real hardware ². It works on Windows, macOS, Linux (X11).
- **Overlay UI (egui or similar)** – (Optional) If we want a Raycast/Spotlight-style prompt overlay for AI commands, a lightweight GUI like `egui` can display a quick text input interface and suggestions. This isn’t strictly necessary but can enhance shortcuts.

Each crate is well-tested: `tray-icon` ensures native look/feel ³, `global_hotkey` cleanly handles cross-platform key events ⁴, `cpal` is the go-to audio I/O lib, and `enigo` robustly simulates keyboard/mouse events ². `Reqwest` handles the network calls needed for APIs ⁶. Together they form a powerful “Rust-Windows” toolkit.

2. High-Level Workflow

1. **Idle (Tray & Waiting):** The app runs in the system tray with an event loop. It registers a global hotkey (e.g. Alt+Space). On Windows, tray icons and hotkeys require a Win32 message loop (handled by `winit`/`tao` under the hood). As the `tray-icon` docs note, “on Windows and Linux, an event loop must be running on the thread... On macOS, an event loop must be running on the main thread” ⁷. In practice, using `tray-icon` + a GUI/event-loop crate takes care of this.
2. **Trigger (Hotkey Event):** When the user presses the hotkey, the app marks “recording mode” true and starts capturing audio via `cpal`. We open a `cpal::Stream` on the default microphone and push incoming samples into an in-memory buffer. (We might play a brief tone or show a tray notification to signal “recording”).
3. **Stop (Release Hotkey):** When the hotkey is released, we stop the stream. We now have raw audio samples in memory (e.g. 16-bit PCM). We write these to a WAV file (using `hound::WavWriter`). WAV is required because Google’s Speech API (and OpenAI’s Whisper endpoint) accept common audio formats like WAV or MP3. As the `hound` crate docs state, it can “read and write the WAVE audio format” ⁵, making it ideal for encoding our buffer to `Recording.wav`.
4. **Transcription (API Call):** Now send the WAV to a speech-to-text service. The user asked to “support Google as the transcription provider if possible, OpenRouter if not.” So by preference:
5. **Google Speech-to-Text:** We use Google Cloud’s STT API (the *Chirp* model, v2). For example, POST `Recording.wav` (or its bytes) to Google’s REST endpoint (with proper API key/credentials). Google’s service will return a JSON transcript. (Google supports 85+ languages ⁸, but here we’ll set `languageCode="en-US"` to restrict to English.) The Google Cloud docs advertise that the API “Convert[s] audio into text transcriptions” ⁸.
6. **OpenAI Whisper (via OpenRouter or direct):** If Google credentials aren’t available or user prefers, we fall back to OpenAI’s API or OpenRouter. OpenAI’s audio API uses the `/v1/audio/transcriptions` endpoint: e.g. `curl --url https://api.openai.com/v1/audio/transcriptions --form file=@/path/Recording.wav --form model=whisper-1` ⁹. This returns the transcript text. (We’d include our own API key.) Alternatively, we can use **OpenRouter’s** normalized API. OpenRouter lets us send base64-encoded audio to a chat-model with `input_audio`. For example, package the WAV as Base64 and call `/api/v1/chat/completions` with a message like `{"type": "input_audio", "inputAudio": {"data": "<base64>", "format": "wav"}}` plus a “transcribe this” prompt ¹⁰. The OpenRouter docs confirm it “supports sending audio files to compatible models via the API” ¹⁰.

In either case, we parse the returned text from JSON. We should also implement retries/fallback (e.g. if Google fails, try Whisper). We limit this to English (as requested) by specifying `language="en-US"` in Google or using the English-only Whisper model.

1. **Type/Paste Result:** Once we have the clean text, we simulate typing it. Using `Enigo`, we call `enigo.text(&transcript)` to send keystrokes to the active window ². This will “type” the words as if the user had pressed them. Enigo works on Windows, but note a key quirk: on Windows, an application running as Administrator *cannot* send input to a lower-privilege target. So our app

should run at normal privilege if, say, Notepad is normal user. (Running as admin while typing into a normal window will silently fail.) Once typing is done, we can optionally show a system notification or tray feedback.

This workflow (hotkey → record → API → type) can be implemented as an event loop. For example, a rough outline is in the prompt's pseudocode: on HotKey Press start `cpal` stream, on Release stop and call transcription, then `enigo.text(&output)` ⁴ ².

3. Detailed Implementation Notes

- **Global Hotkey Loop:** We'll use `global_hotkey::GlobalHotKeyManager` to register something like `Alt+Space`. That crate's example shows creating a `HotKey` with modifiers+code ⁴. We run an event loop (e.g. polling `GlobalHotKeyEvent::receiver()`) to detect press vs release. Crucially, on Windows the hotkey callbacks will be delivered in the Win32 message loop – using `winit` or `tao` as our window/event backend handles this automatically, as noted in the *tray-icon* documentation ⁷.
- **Recording Audio (cpal):** When recording starts, create a new `cpal::Stream` on the default input device. The callback collects samples (e.g. `&[i16]` or `&[f32]`) into a `Vec`. When stopping, we finalize writing: use `hound::WavWriter::create("temp.wav", spec)` and `writer.write_sample(...)` for each sample. After finishing, `writer.finalize()` produces a valid `temp.wav`. (Alternatively, write to an in-memory buffer and use `request` with a multipart form to avoid disk. But disk temp is simpler.) The `hound` crate is specifically for this: it “renders audio and stores it as a WAV file” ⁵, which is exactly what we need.
- **Sending to Google Speech:** Assuming Google STT is chosen, use `request::Client` to POST. Google's v2 API expects JSON with base64-encoded audio or raw bytes. For example, one can do a multipart POST or JSON body with `"content": "<base64 of audio>"` and `"languageCode": "en-US"`. (We must set up Google credentials beforehand.) The response JSON will include `"transcript"`. All requests should be async (or blocking client with tokio).
- **Sending to OpenAI (Whisper):** If using OpenAI directly, we do a multipart upload to `https://api.openai.com/v1/audio/transcriptions`, include our audio file and model. The example from OpenAI's docs ⁹ shows the exact curl. In Rust, we can use `request` to mimic that: `client.post("https://api.openai.com/v1/audio/transcriptions").header("Authorization", "Bearer ...").multipart(form_with_file_and_model).send()`. Parse the JSON for `text`.
- **Sending via OpenRouter:** If using OpenRouter instead, follow their “Sending Audio Files” guide. We base64-encode the WAV and send to `/api/v1/chat/completions` with messages containing the audio part ¹⁰. OpenRouter's JSON schema requires `"messages": [{"role": "user", "content": [{"type": "text", "text": "Please transcribe this audio."}, {"type": "input_audio", "inputAudio": {"data": "...base64...", "format": "wav"}}]}`. The response will have the transcript. OpenRouter abstracts away provider details, so you can plug in Whisper, a PaLM2 model, etc., behind the scenes.

- **Typing the Transcript:** Once we have the string `transcribed_text`, we call `enigo.text(&transcribed_text)` ². Enigo will emit key presses (including spaces, punctuation, etc.). If more control is needed (like simulating enter or tab), Enigo supports methods like `key(Key::Return, Press)` and `release`. But simple `.text()` handles Unicode text well.
- **System Tray & UI:** Using `tray-icon`, we create a tray icon with a tooltip (e.g. “Rust Voice Typing”). We can attach menu items (like “Settings” or “Quit”) and listen for click events. We should run `TrayIconEvent::receiver().try_rcv()` in our loop to process clicks (see [14†L371-L379]). This is purely cosmetic but gives users a way to exit the app or open preferences. The tray-icon docs note we get events via `TrayIconEvent::receiver()` ¹¹.
- **Overlay Prompt (Optional):** For Raycast-style quick commands (“Fix Grammar”, “Translate to Spanish”, etc.), we could display an `egui` window on hotkey or a separate shortcut. This window would let the user type a prompt. Internally, we’d capture selected text via clipboard (see below) and send it + prompt to an LLM (OpenAI or OpenRouter chat completion). The concept is: user highlights text, hits a different hotkey (say Ctrl+Alt+Space), our app reads the clipboard, sends an AI prompt, and pastes the result.

4. Windows-Specific Considerations

- **Message Loop:** As noted, Windows needs a Win32 message loop for hotkeys and tray interactions ⁷. If we use a GUI/event library (even headless via `winit` or just `tauri-utils`), it handles that. Do *not* spin CPU tightly; use a small sleep or event-blocking.
- **Audio Formats:** OpenAI/Whisper expects formats like WAV, MP3, M4A, OGG, etc. ⁹. We’ll stick with WAV (linear PCM) since it’s lossless and easy to encode with `hound`.
- **Admin vs User Rights:** On Windows, a process running as Administrator cannot send simulated inputs (via Enigo) to a lower-privileged window (due to UIPI). So to ensure typing works in e.g. Notepad, either run both at same level or run as non-admin. In practice, don’t “Run as Administrator” for this app unless needed.
- **Clipboard Shortcuts (Raycast style):** For advanced “fix grammar” shortcuts: use Enigo to simulate `Ctrl+C`, then use a clipboard crate like `arboard`. Arboard is “a cross-platform library for interacting with the clipboard” ¹². In code, do `enigo.key(Key::Control, Press); enigo.key(Key::Unicode('c'), Press); ...; enigo.key(Key::Control, Release);` to copy the selection. Then `let clip_text = arboard::Clipboard::new()?.get_text()?;`. You send `clip_text` + your prompt (e.g. “Fix grammar: ...”) to an LLM. When you get the result, simulate paste with `enigo.key(Key::Control, Press); enigo.key(Key::Unicode('v'), Click); enigo.key(Key::Control, Release);` ². This covers the Ottex-style shortcut flow: *copy selection, transform via AI, paste back*.
- **Error Handling & UX:** Include retry logic and user feedback. For example, if transcription fails, show a tray notification or second pressing of hotkey to copy the raw buffer to file. Also consider showing an overlay “Listening...” or “Processing...” cue. Ensure hotkey state resets properly if something goes wrong.

5. “Raycast” Style AI Shortcuts (Fix Grammar, etc.)

Beyond voice-to-text, Ottex also offers AI transformations on selected text. We can implement similar one-click shortcuts:

- **Read Selection:** On a separate hotkey (e.g. Ctrl+Shift+R), simulate **Copy** (Ctrl+C) as above. Use `arboard` to retrieve the selected text from the clipboard ¹².
- **Process with LLM:** Prepend or append an instruction (e.g. “Fix grammar:” or “Translate to Spanish:”). Send this string to a chat API (either OpenAI or OpenRouter). For example, call OpenRouter’s chat completion with a system/user message.
- **Replace Text:** Take the LLM’s response text and simulate **Paste** (Ctrl+V). This replaces the selection with the transformed text.

This flow is essentially: *read clipboard* → *LLM prompt* → *write clipboard*. Using `enigo` and `arboard` covers the keyboard/clipboard steps ² ¹². It sidesteps needing a GUI context menu. Optionally, one could overlay a GUI prompt (via `egui`) to let the user choose the shortcut or edit the prompt, but it’s not strictly necessary.

6. Summary

In summary, the Rust app uses a combination of low-level system hooks and high-level AI APIs:

- **Crates:** `tray-icon` for the tray UI ³, `global_hotkey` for hotkeys ⁴, `cpal` for audio, `hound` for WAV, `request` for HTTP, `enigo` for input simulation ², and `arboard` for clipboard ¹².
- **Flow:** On hotkey, record mic → save WAV (with `hound`) → send to Google STT or OpenAI/Whisper API (via HTTP) → get transcript → simulate typing with `enigo`. For AI text shortcuts, simulate copy via `enigo`, use `arboard` to fetch text, send to LLM, then paste the result ² ¹².
- **Windows Nuances:** Use a proper Win32 event loop (handled by `tray-icon` / `tao`) ⁷, and avoid privilege mismatches when typing.

By following this design, you’ll have a foundation to code the tool: the outlined crates and API calls cover all core features. The end result will be a Windows equivalent of Ottex.ai’s voice-to-text and AI-shortcuts, built with Rust’s safety and performance.

Sources: This plan is based on Ottex.ai’s description ¹ and docs, along with the cited Rust crate documentations and API guides ³ ² ⁴ ⁶ ⁸ ⁹ ⁵ ¹².

¹ Ottex AI - Smart AI Voice to Text
<https://ottex.ai/>

² `enigo` - Rust
<https://docs.rs/enigo/latest/enigo/>

³ ⁷ ¹¹ GitHub - tauri-apps/tray-icon: Tray icons for Desktop Applications in Rust.
<https://github.com/tauri-apps/tray-icon>

4 global_hotkey::hotkey - Rust

https://docs.rs/global-hotkey/latest/global_hotkey/hotkey/index.html

5 hound - Rust

<https://docs.rs/hound/latest/hound/>

6 reqwest - Rust

<https://docs.rs/reqwest/latest/reqwest/>

8 Speech-to-Text API: speech recognition and transcription | Google Cloud

<https://cloud.google.com/speech-to-text>

9 Speech to text | OpenAI API

<https://platform.openai.com/docs/guides/speech-to-text>

10 OpenRouter Audio Inputs | Complete Documentation | OpenRouter | Documentation

<https://openrouter.ai/docs/guides/overview/multimodal/audio>

12 GitHub - 1Password/arboard: A clipboard for Rust

<https://github.com/1Password/arboard>